

CodeGen Project Plan v6 — Dual-Track: Text-to-SQL + Text-to-NoSQL

Grounded in: SchemaRAG (SIGMOD 2026) · TEND (Text-to-NoSQL) · Project Proposal

Version history: v5 → v6 updates SchemaRAG codebase audit findings, actual implementations, and new src/ files

v6.1 patch: Phase 8A bug audit — 3 issues found and fixed in build_cot_data.py

SECTION 1 — PROJECT OVERVIEW

What We Are Building

A unified natural-language-to-query system with two operational tracks:

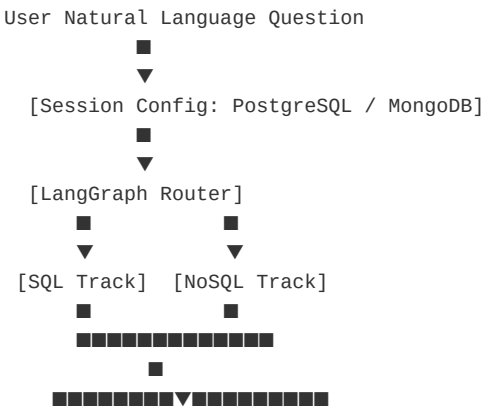
Track	Input	Output	Target DB	Primary Paper
Text-to-SQL	Natural language	PostgreSQL/SQLite SQL	PostgreSQL	SchemaRAG (SIGMOD 2026)
Text-to-NoSQL	Natural language	MongoDB MQL	MongoDB	TEND / SMART

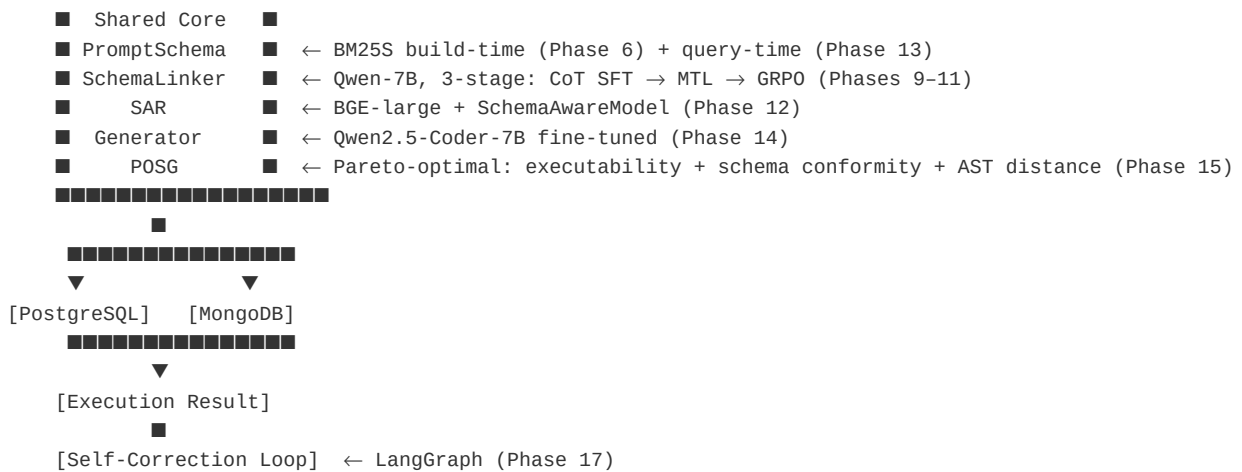
Both tracks share a common backbone (SchemaLinker → SAR → Generator → POSG) unified under a LangGraph orchestration layer.

Routing Strategy

- **Session-based:** User selects PostgreSQL or MongoDB at session start
- **Fallback detection:** If input looks like an existing SQL query (SELECT/INSERT/UPDATE), route to SQL-to-NoSQL migration utility (CP4)

Architecture Diagram





NOTE on "Shared" vs "Split"

Architecture is shared; models are trained SEPARATELY per track:

- SchemaLinker SQL checkpoint ≠ SchemaLinker NoSQL checkpoint
- SAR SQL index ≠ SAR NoSQL index
- Generator SQL fine-tune ≠ Generator NoSQL fine-tune

This is deliberate — the TEND paper shows direct generation outperforms cascade by 20+ points precisely because NoSQL queries have fundamentally different structure.

SECTION 2 — HARDWARE STRATEGY

Work	Where	Why
Data prep, FK graphs, BM25S, MongoDB conversion	Mac M1	No GPU needed
CoT generation + MQL translation (API calls)	Mac M1	Network I/O, no GPU
SchemaLinker Stage 1 SFT	Colab T4 (free)	16GB fits Qwen-7B with LoRA r=64
SchemaLinker Stage 2 MTL	Colab T4 (free)	Same
SchemaLinker Stage 3 GRPO	Colab A100 (Pro)	G=8 samples × Qwen-7B ≈ 28GB min
SAR training	Colab T4 (free)	BGE-large encoder + SchemaAwareModel
Qwen2.5-Coder-7B SFT (Generator)	Colab A100 (Pro)	7B + LoRA needs 24GB+ for batch
Inference / pipeline testing	Mac M1 (4-bit GGUF) or Colab T4	
LangGraph assembly, POSG, demo	Mac M1	No GPU

Checkpoint strategy: All Colab-trained models saved to Google Drive at </content/drive/MyDrive/codegen/checkpoints/>.

Mac ↔ Colab workflow: Write training scripts on Mac → git push → git pull on Colab → train → save to Drive.

SECTION 3 — PHASE STATUS

Phase	Description	Status
1	Project planning and literature review	■ Done
2	Architecture design decisions	■ Done
3A–3E	Environment setup, project structure, device helper	■ Done
4	Spider dataset — 7000 Q-SQL pairs + 166 SQLite databases	■ Done
5A	FK graph builder — NetworkX DiGraph for all 166 databases	■ Done
5B	MongoDB converter — all 166 databases converted and verified	■ Done
6	PromptSchema — BM25S column annotations (SQL + NoSQL)	■ Done
7A	SQL RAG corpus — 7000 Q-SQL pairs, 57 structural types, 7-dim vector	■ Done
7B	NoSQL RAG corpus — Q-MQL via DeepSeek + MongoDB verification	■ In progress
8A	SQL CoT data — adapted from SchemaRAG script_to_COT.py	■ In progress
8B	NoSQL CoT data	■ Blocked on 7B
9A–20	SchemaLinker training, SAR, Generator, POSG, eval, demo	■ Pending

SECTION 4 — SCHEMARAG CODEBASE AUDIT (NEW IN V6)

What SchemaRAG Released

We cloned the SchemaRAG GitHub repo to [external/SchemaRAG/](#) and audited all scripts. Key findings:

File	Released?	What it contains	Action taken
datas/RAG_Spider.json	■ Yes	3102 Q-SQL pairs with formatted schema text (our Phase 7A output equivalent)	Kept our 7000-entry version; theirs is curated subset
datas/RAG_BIRD.json	■ Yes	3835 Q-SQL pairs for BIRD benchmark	Available for future extension
datas/UniSQL.json	■ Yes	1034 Q-SQL pairs (university internal dataset)	Not used
CoT training data	■ Not released	script_to_COT.py provided but not the generated data	Must generate via Phase 8A
MTL error dataset	■ Not released	SchemaLinker_train/find_mistakes.py provided	Must generate via Phase 10A

Scripts We Adapted

All SchemaRAG scripts were reviewed and adapted into our [src/](#) structure. The following table shows what was borrowed and why:

SchemaRAG script	Our file	Key changes
<code>llm_local.py</code>	<code>src/model_interface.py</code>	<code>modelscope</code> → <code>transformers</code> ; MPS/CPU/CUDA via <code>src/device.py</code> ; configurable params
<code>function.py</code> → <code>extract_db_samples_enriched_bm25</code>	<code>src/schema_utils.py</code>	UTF-8 fix; evidence defaults <code>""</code> ; question-aware BM25 for inference
<code>SchemaLinker_fix.py</code>	<code>src/schema_linker/fix.py</code>	Removed hardcoded paths; takes <code>schema_text</code> string; lazy <code>FlagEmbedding</code> import
<code>use_SchemaLinker.py</code>	<code>src/schema_linker/infer.py</code>	Uses <code>ModelInterface</code> ; evidence removed (Spider); retry loop kept exactly
<code>train_SchemaLinker_CoT_peft.py</code>	<code>src/schema_linker/train_stage1.py</code>	Input format → our <code>sql_cot_train.json</code> ; LoRA <code>r=64</code> (raised from paper's <code>r=16</code>)
<code>train_SchemaLinker_MTL_peft.py</code>	<code>src/schema_linker/train_stage2.py</code>	<code>deepspeed</code> removed; <code>argparse</code> paths; <code>WeightedRandomSampler</code> kept
<code>train_SchemaLinker_GRP0_peft.py</code>	<code>src/schema_linker/train_stage3_grpo.py</code>	Reward function ported exactly (TP/FP/FN config); TRL <code>GRPOTrainer</code>
<code>train_SAR.py</code> (SchemaAwareModel)	<code>src/sar/sar_model.py</code>	Architecture + NaN guards preserved exactly; moved to own file
<code>train_SAR.py</code> (training loop)	<code>src/sar/train.py</code>	Embedding cache + contrastive triplet loss; <code>FlagModel</code> configurable
<code>SAR_use.py</code>	<code>src/sar/infer.py</code>	Clean <code>SARRetriever</code> class; pre-computes corpus embeddings at load
<code>SAR_train/format_schema.py</code>	<code>src/sar/format_schema.py</code>	Parses our schema text format; adds <code>parsed_schema</code> field
<code>po.py</code>	<code>src/posg/posg_sql.py</code>	<code>validate_sql_statement</code> → direct SQLite execute; hardcoded paths removed
<code>po.py</code> (adapted)	<code>src/posg/posg_nosql.py</code>	MQL-specific: pipeline stage-type similarity replaces AST edit distance
<code>eval/exec_eval.py</code>	<code>src/eval/exec_eval.py</code>	Async removed; UTF-8 fix; clean <code>evaluate_ex()</code> public API
<code>script_to_CoT.py</code>	<code>scripts/build_cot_data.py</code>	<code>DeepSeek</code> replaces <code>GPT-4o</code> ; <code>sqlglot</code> entity validation replaces second LLM call; our Spider data format

Key Design Differences vs SchemaRAG

Component	SchemaRAG approach	Our approach	Why we differ
BM25S query	Question text at inference time	Column name at build time	Training pipeline efficiency; inference uses <code>src/schema_utils.py</code> (their approach)
SQL parser (RAG corpus)	<code>sqlparse</code>	<code>sqlglot</code>	<code>sqlglot</code> parsed all 7000 Spider SQLs with 0 failures; typed AST nodes
SQL parser (POSG AST)	<code>sqlparse</code>	<code>sqlparse</code>	AST edit distance works on <code>sqlparse</code> 's token trees; no change here
Structural type vector	6 dimensions	7 dimensions (added <code>has_set_op</code>)	UNION/INTERSECT/EXCEPT have fundamentally different structure from plain SELECT

Component	SchemaRAG approach	Our approach	Why we differ
CoT format	<reasoning> tags	**<think> tags**	Adopted from SchemaRAG's updated <code>script_to_CoT.py</code> which uses <code><think></think></code>
CoT entity validation	Second LLM call (<code>extract_sql_entities</code>)	sqlglot	Free, no extra API cost, more reliable for Spider SQL patterns
Teacher model for CoT	GPT-4o	DeepSeek-V3	~10× cheaper; comparable quality on structured CoT tasks
LoRA rank	r=16 (Stage 1 paper)	r=64	Higher rank → better capacity for complex reasoning; T4 still fits at bf16

SECTION 5 — ACTUAL FILE STRUCTURE (AS BUILT)

```

Codegen/
■■■ src/
■   ■■■ device.py           ■ Phase 3D
■   ■■■ fk_graph.py        ■ Phase 5A
■   ■■■ prompt_schema.py   ■ Phase 6 (build-time BM25S)
■   ■■■ schema_utils.py    ■ NEW – query-time BM25S (SchemaRAG function.py)
■   ■■■ model_interface.py  ■ NEW – Qwen inference wrapper (SchemaRAG llm_local.py)
■   ■■■ mongodb_converter.py ■ Phase 5B
■   ■■■ schema_linker/
■   ■   ■■■ train_stage1.py ■ NEW – CoT SFT (SchemaRAG train_SchemaLinker_CoT_peft.py)
■   ■   ■■■ train_stage2.py ■ NEW – MTL (SchemaRAG train_SchemaLinker_MTL_peft.py)
■   ■   ■■■ train_stage3_grpo.py ■ NEW – GRPO (SchemaRAG train_SchemaLinker_GRPO_peft.py)
■   ■   ■■■ infer.py        ■ NEW – inference + retry (SchemaRAG use_SchemaLinker.py)
■   ■   ■■■ fix.py          ■ NEW – embedding-based link correction (SchemaRAG SchemaLinker_fix.py)
■   ■■■ sar/
■   ■   ■■■ sar_model.py    ■ NEW – SchemaAwareModel + SafeMultiheadAttention (SchemaRAG train_SAR.py)
■   ■   ■■■ train.py        ■ NEW – contrastive training loop (SchemaRAG train_SAR.py)
■   ■   ■■■ infer.py        ■ NEW – SARRetriever class (SchemaRAG SAR_use.py)
■   ■   ■■■ format_schema.py ■ NEW – schema text parser (SchemaRAG SAR_train/format_schema.py)
■   ■■■ generator/
■   ■   ■■■ train.py        ■ Phase 14 (stub)
■   ■   ■■■ infer.py        ■ Phase 16 (stub)
■   ■■■ posg/
■   ■   ■■■ posg_sql.py     ■ NEW – Pareto-optimal SQL selector (SchemaRAG po.py)
■   ■   ■■■ posg_nosql.py   ■ NEW – Pareto-optimal MQL selector (adapted from po.py)
■   ■■■ eval/
■   ■   ■■■ exec_eval.py    ■ NEW – EX metric, permutation-aware (SchemaRAG eval/exec_eval.py)
■   ■■■ router/
■   ■   ■■■ langgraph_router.py ■ Phase 17 (stub)
■■■ scripts/
■   ■■■ validate_spider.py  ■ Phase 4
■   ■■■ validate_sql2mongo_conversion.py ■ Phase 5B
■   ■■■ build_rag_corpus.py  ■ Phase 7A
■   ■■■ build_nosql_rag_corpus.py ■ Phase 7B (running)
■   ■■■ build_cot_data.py    ■ Phase 8A (running)
■■■ Data/
■   ■■■ Spider/
■   ■   ■■■ fk_graphs/      ■ 7000 Q-SQL + 166 SQLite DBs
■   ■   ■■■ mongodb/        ■ 166 JSON files
■   ■   ■■■ prompt_schema/sql/ ■ 166 schema JSONs + live MongoDB
■   ■   ■■■ prompt_schema/nosql/ ■ 166 JSON files
■   ■   ■■■ rag_corpus/
■   ■   ■■■ spider_sql_rag.json ■ 7000 entries, 57 types, 7-dim vector
■   ■   ■■■ spider_nosql_rag.json ■ generating (target 4000-5000)

```

■ ■ ■■ nosql_checkpoint.json	■ checkpoint file
■ ■■ cot_data/	
■ ■■ sql_cot_train.json	■ generating (target 4000-5000)
■ ■■ cot_checkpoint.json	■ checkpoint file
■■■ external/	
■ ■■ SchemaRAG/	■ Cloned; all scripts audited
■■■ docs/	
■ ■■ architecture.md	■ Comprehensive reference through Phase 8A
■ ■■ SchemaRAG.pdf	
■ ■■ Text_to_NoSQL.pdf	
■■■ configs/	
■ ■■ config.yaml	
■■■ CodeGen_Plan_v6_DualTrack.md	← this file

SECTION 6 — PHASE DETAILS (COMPLETE)

Track annotations:

- ■ = Shared foundation
 - ■ = SQL-only
 - ■ = NoSQL-only
 - ■■ = Done separately per track
-

PHASE 3D–3E — Environment + Project Structure [■] ■ DONE

`src/device.py` created. Full directory tree, `configs/config.yaml`, `.gitignore`, `.env` created and committed.

PHASE 4 — Spider Dataset [■] ■ DONE

7000 `train_spider.json` entries, 1034 `dev.json` entries, 166 SQLite databases. Validated via `scripts/validate_spider.py`.

PHASE 5A — FK Graph Builder [■] ■ DONE

`src/fk_graph.py` — NetworkX DiGraph for all 166 databases. In-degree centrality identifies bridge/central tables. Output cached to `Data/fk_graphs/{db_name}.json`.

Actual implementation notes:

- Uses `PRAGMA foreign_key_list` for FK extraction
 - Uses `PRAGMA table_info` for column/PK info
 - SQLite UTF-8 error handling not needed here (metadata only)
-

PHASE 5B — MongoDB Converter [■] ■ DONE

`src/mongodb_converter.py` — SQLite → MongoDB with:

- Type coercion: `str→int→float→str` cascade
 - UTF-8 fix: `conn.text_factory = lambda b: b.decode("utf-8", errors="replace")`
 - v1 strategy: all tables → separate collections (reference-based FK, not embedded)
 - Validated by `scripts/Validate_sql2mongo_conversion.py` — all 166 databases, row counts match perfectly
-

PHASE 6 — PromptSchema via BM25S [■] ■ DONE

`src/prompt_schema.py` — **build-time** BM25S annotation for all 166 databases (SQL + NoSQL).

Design: Column name used as BM25 query at build time → cached JSON. At inference time, `src/schema_utils.py` runs question-aware BM25 (SchemaRAG's approach) to replace cached values with question-relevant samples.

****Key difference from SchemaRAG's `BM25s_constrcut_db.py`**:**

- SchemaRAG runs BM25S per-query at inference (question as query)
- We run BM25S once at build time (column name as query)
- Our `src/schema_utils.py` provides SchemaRAG's inference-time approach for Phase 13+

PHASE 7A — SQL RAG Corpus [■] ■ DONE

`scripts/build_rag_corpus.py` — 7000 entries, 0 parse failures, 57 unique structural types.

Actual structural type vector (7 dimensions, not 6 as planned in v5):

```
{
  "num_joins":    int,          # 0, 1, 2, 3+
  "num_tables":  int,
  "has_group_by": bool,
  "has_order_by": bool,
  "has_having":  bool,
  "has_subquery": bool,
  "has_set_op":  bool,          # ← NEW: UNION/INTERSECT/EXCEPT
}
```

`has_set_op` was added because UNION/INTERSECT/EXCEPT queries are structurally incompatible with plain SELECT queries — pairing them as positives in SAR contrastive training would be incorrect.

Parser choice: `sqlglot` (not `sqlparse`) — parsed all 7000 SQLs with 0 failures; provides typed AST nodes for reliable structural analysis. (`sqlparse` is still used in `posg_sql.py` for AST edit distance, where its token-tree structure is more suitable.)

****SchemaRAG released `RAG_Spider.json`**** (3102 entries). We kept our 7000-entry corpus — more data, richer structural type annotations.

PHASE 7B — NoSQL RAG Corpus [■] ■ IN PROGRESS

`scripts/build_nosql_rag_corpus.py` — translates Q-SQL pairs to Q-MQL pairs via DeepSeek-V3 API, verifies by executing on MongoDB vs SQLite.

Test result (5 entries before full batch): 9/10 passed. The one failure was an AVG aggregation query where MongoDB returned 0 docs — known limitation of count-based result comparison.

Output format:

```
{
  "question":      "How many singers are from France?",
  "mql_collection": "singer",
  "mql_pipeline":  [{"$match": {"Country": "France"}}, {"$count": "total"}],
  "db_name":       "concert_singer",
  "structural_type": {...},
  "source_sql":    "SELECT COUNT(*) FROM singer WHERE Country = 'France'"
}
```

Target: 4000–5000 verified pairs. Output: `Data/rag_corpus/spider_nosql_rag.json`.
Cost: ~\$1.68 (7000 × 800 tokens × \$0.0003/1K).

PHASE 8A — SQL SchemaLinker CoT Data [■] ■ IN PROGRESS

`scripts/build_cot_data.py` — adapted from SchemaRAG's `script_to_COT.py`.

CoT format (adopted from SchemaRAG, not the `<reasoning>` format in v5):

```
<think>
1. Understand the key concepts in the question:
  • ...
2. Analyze database table relationships:
  • ...
3. Key field for filtering: **table.column** (why this field is critical)
</think>

Summary paragraph...

The key field matching the question is: [table.column].
```

Validation pipeline (2 steps):

- 1. **Format check:** `<think>` tags present, 3 steps, final key declaration
- 2. **Entity check:** key field's table must appear in SQL tables (via sqlglot, not a second LLM call)

Test result: 4/5 passed on first run. The 1 failure was a no-WHERE-clause query (`SELECT * FROM teams`) — correctly filtered out since there is no key filtering field.

Schema formatting: Uses `format_schema_text()` from `build_cot_data.py` which combines FK graph (PK info) + PromptSchema (sample values) into SchemaRAG's text format:

```
# Table: actor
[(actor_id:INT, Primary Key, Examples: [1, 2]),
 (name:TEXT, Examples: [Tom Hanks, Meryl Streep]),
]
# Foreign Keys:
# actor_in_movie.actor_id -> actor.actor_id
```

Checkpointing: Every 50 entries to `Data/cot_data/cot_checkpoint.json` — resumes from last checkpoint on restart.

Output: `Data/cot_data/sql_cot_train.json`. Target: 4000–5000 verified CoT examples.

Bugs found and fixed (post-v6 audit):

#	Bug	Impact	Fix
1	Checkpoint triggered every 100 entries, not 50 as documented	Up to 50 lost API calls on crash	Fixed: % 50 on line 317
2	<code>validate_format</code> used unanchored <code>re.search</code> ; extraction patterns anchored to \$	Mismatch: text like <code>[head.age] (extra note)</code> passes format check but extraction returns <code>{}</code> , causing the entry to be counted under <code>entity_fail</code> instead of <code>format_fail</code>	Fixed: <code>validate_format</code> now uses the same anchored pattern (<code>\s*\$</code>) with <code>re.MULTILINE</code> as the extraction functions
3	<code>\[?</code> and <code>\]?</code> in validation pattern are independently optional	<code>[head.age</code> (open bracket, no closing) passes validation; extraction handles it correctly but stat counts mislabel it	Fixed by sharing a single pattern across all three uses — unbalanced brackets are now handled identically

Note: Bug 2 and 3 did NOT cause incorrect data to be saved — `validate_entities` caught those entries and rejected them correctly. The impact was misleading stats (`entity_fail` count inflated, `format_fail` count deflated), not bad training data.

PHASE 8B — NoSQL SchemaLinker CoT Data [■] ■ BLOCKED ON 7B

Same process as 8A but for MongoDB schemas. Blocked until Phase 7B produces `spider_nosql_rag.json`.

Input: Q-MQL pairs from Phase 7B (~4000–5000)

Output format: `<think>` + 3-step reasoning + The key field matching the question is: `[collection.field]`

Output: `Data/cot_data/nosql_cot_train.json`

PHASE 9A — SchemaLinker Stage 1: SQL CoT SFT [■, Colab T4]

Fine-tune Qwen-7B on `sql_cot_train.json`. Script: `src/schema_linker/train_stage1.py`.

LoRA config (updated from v5 — `r` raised from 16 to 64):

```
LoraConfig(
    r=64, lora_alpha=32,
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj",
        "gate_proj", "up_proj", "down_proj"],
    lora_dropout=0.1, bias="none", task_type="CAUSAL_LM"
)
```

Input/output format (actual, using Qwen chat template):

```
<|im_start|>system
You are a Schema Linking Expert<|im_end|>
<|im_start|>user
# Question:{question}
# Database:{schema_text}<|im_end|>
<|im_start|>assistant
<think>{reasoning}</think>{key_field_declaration}
```

Training: 3 epochs, batch=4, gradient accumulation=4 (effective 16), LR=2e-4, max_len=2048, bf16, cosine schedule, early stopping patience=2.

Run:

```
python -m src.schema_linker.train_stage1 \
    --data Data/cot_data/sql_cot_train.json \
    --model Qwen/Qwen2.5-7B \
    --out models/schema_linker_cot
```

Save: `Drive/codegen/checkpoints/sl_sql_stage1/`

Target: Table Recall > 0.85, Column Recall > 0.70

PHASE 9B — SchemaLinker Stage 1: NoSQL CoT SFT [■, Colab T4]

Same as 9A on `nosql_cot_train.json`. Initialize from SQL Stage-1 checkpoint for transfer learning.

Save: `Drive/codegen/checkpoints/sl_nosql_stage1/`

PHASE 10A — SchemaLinker Stage 2: SQL Multi-Task Learning [■, Colab T4]

Script: `src/schema_linker/train_stage2.py`.

Input data format (each item needs additional error fields from error mining):

```
{
  "question":      "...",
  "database":      "schema text",
  "think":         "correct reasoning",
  "answer":         "The key field matching...",
  "think_pre":     "wrong reasoning from Stage-1",
  "schema_links_pred": ["wrong.prediction"],
  "error_explanation": "The model incorrectly identified..."
}
```

3 tasks trained jointly:

- Task 0 — error_detection (weight 0.3)
- Task 1 — correction (weight 0.4)
- Task 2 — generation (weight 1.0)

WeightedRandomSampler balances task distribution proportional to inverse frequency × task weight.

Error dataset construction:

1. Run Stage-1 model on all 7000 Spider train × inference
2. Collect predictions ≠ ground truth → candidate error set
3. Filter with DeepSeek: only keep cases where wrong schema → wrong SQL
4. Target: ~500–800 curated error examples → save as `Data/cot_data/mtl_train.json`

Save: `Drive/codegen/checkpoints/sl_sql_stage2/`

PHASE 10B — SchemaLinker Stage 2: NoSQL MTL [■, Colab T4]

Same as 10A on NoSQL error data. Initialize from `sl_nosql_stage1`.

Save: `Drive/codegen/checkpoints/sl_nosql_stage2/`

PHASE 11A — SchemaLinker Stage 3: SQL GRPO [■, Colab A100]

Script: `src/schema_linker/train_stage3_grpo.py`.

Reward function (ported exactly from SchemaRAG):

```
reward = (
    +2.0 × true_positives    # TP: correctly predicted key fields
    - 0.5 × false_positives  # FP: predicted but not in ground truth
    - 3.0 × false_negatives  # FN: missed required key fields (hardest penalty)
    + 0.5 × f1_score         # F1 bonus
)
# Format failure: -1000 (ensures format compliance is non-negotiable)
```

Why FN penalty is strongest: A missing required table makes correct SQL impossible. An extra table is tolerable — the decoder can ignore it.

GRPO config: 4 candidate generations per prompt, LR=5e-6, bf16.

Target metrics (from SchemaRAG Appendix B, k=5):

- Table Recall > 0.96
- Column Recall > 0.80
- Column MCC > 0.85

Save: `Drive/codegen/checkpoints/sl_sql_final/`

PHASE 11B — SchemaLinker Stage 3: NoSQL GRPO [■, Colab A100]

Same reward function — FN is equally harmful for MQL (missing a collection = incorrect pipeline).

Post-training fix: Use `src/schema_linker/fix.py` (adapted from SchemaRAG `SchemaLinker_fix.py`) to snap predicted links to nearest real `collection.field` via BGE embedding cosine similarity. This corrects hallucinations like `actor.nationality` → `actor.country`.

Save: `Drive/codegen/checkpoints/sl_nosql_final/`

PHASE 12A — SAR Training for SQL [■, Colab T4]

Scripts: `src/sar/sar_model.py` + `src/sar/train.py`.

SAR Architecture (ported from SchemaRAG `train_SAR.py`, NaN guards preserved):

Input: BGE-large embeddings (dim=1024) for question, tables, columns

Stage 1 – Column-aware table embeddings (table_column_attention):

```
T^C_i = SafeMultiheadAttention(query=T_i, key=C_i, value=C_i)
T^C_i = LayerNorm(T^C_i + T_i)
```

Stage 2 – Question-schema fusion (question_table_attention):

```
■ = SafeMultiheadAttention(query=Q, key=T^C, value=T^C)
output = LayerNorm(■ + Q)
output = output_proj(output) → [batch, embed_dim]
```

****SafeMultiheadAttention**:** Handles edge cases where all keys are masked (some databases have tables with no valid columns). Returns zeros for those samples without crashing.

Training loss: Triplet loss (margin=0.3) with positive/negative pairs from `structural_type` grouping.

Run:

```
python -m src.sar.train \
  --corpus Data/rag_corpus/spider_sql_rag.json \
  --out models/sar_sql \
  --epochs 10 --batch 32 --lr 1e-4
```

Save: `Drive/codegen/checkpoints/sar_sql/`

PHASE 12B — SAR Training for NoSQL [■, Colab T4]

Same architecture, smaller dataset. Use `spider_nosql_rag.json`.

Save: `Drive/codegen/checkpoints/sar_nosql/`

PHASE 13 — ChromaDB Index Building [■, Mac]

Build vector stores using trained SAR encoders. At inference time, `src/sar/infer.py` (adapted from SchemaRAG `SAR_use.py`) pre-computes corpus embeddings at load time and retrieves top-k via cosine similarity.

Query-time BM25S: Replace build-time PromptSchema values with question-relevant values using `src/schema_utils.py` → `extract_db_samples_enriched_bm25()`. This uses the actual question as the BM25 query, producing more relevant schema examples.

Save: `indexes/chroma_sql/`, `indexes/chroma_nosql/`

PHASE 14A — SQL Generator Fine-tuning [■, Colab A100]

Fine-tune `Qwen/Qwen2.5-Coder-7B-Instruct` on full pipeline prompts (PromptSchema + SchemaLinker output + SAR top-3 examples).

IMPORTANT: Generate training prompts AFTER Phase 11A (GRPO) — use production SchemaLinker output, not Phase 9A output. Stale SchemaLinker predictions in training prompts would teach the Generator to compensate for wrong schema links.

Save: `Drive/codegen/checkpoints/generator_sql/`

Target: >82% EX on Spider dev (pre-POSG)

PHASE 14B — NoSQL Generator Fine-tuning [■, Colab A100]

Initialize from SQL generator (Phase 14A). Fine-tune on Q-MQL pairs with full NoSQL pipeline context.

Output format: PyMongo aggregation pipeline dict (JSON-serializable, not Python string):
`{"collection": "singer", "pipeline": [{"match": {"Country": "France"}}, {"count": "total"}]}`

Save: `Drive/codegen/checkpoints/generator_nosql/`

Target: >60% EX on NoSQL dev set

PHASE 15A — POSG for SQL [■, Mac]

Script: `src/posg/posg_sql.py` (adapted from SchemaRAG `po.py`).

Algorithm (3 dimensions, Pareto front):

Dimension	How	Notes
Executability	Run on SQLite: 1.0 or 0.0	Hard filter — non-executable candidates excluded from Pareto
Schema conformity	Jaccard(SQL identifiers, predicted schema links)	Uses sqlparse identifier extraction + keyword filter
Example consistency	1 – AST edit distance from retrieved examples	sqlparse AST + tree edit distance algorithm

- **ASTProcessor**** (from SchemaRAG, uses sqlparse):
- Builds typed AST dict recursively from sqlparse tokens
 - Filters whitespace/comment tokens as "non-meaningful"
 - Computes normalized edit distance (distance / max node weight)

Selection strategies: `balanced` (0.5/0.5), `schema_priority` (0.7/0.3), `example_priority` (0.3/0.7).

PHASE 15B — POSG for NoSQL [■, Mac]

Script: `src/posg/posg_nosql.py`.

Key difference from SQL POSG:

Dimension	SQL approach	NoSQL approach
Executability	SQLite execute	MongoDB aggregation with <code>maxTimeMS=3000</code>
Schema conformity	Jaccard over SQL identifiers	Jaccard over collection names (including <code>\$lookup</code> targets)
Example consistency	sqlparse AST edit distance	Pipeline stage-type similarity (<code>\$match</code> , <code>\$group</code> , <code>\$sort</code> sequence comparison)

MQL has no standard AST parser, so stage-type comparison replaces AST edit distance. Two MQL pipelines with the same sequence of `$match` → `$group` → `$sort` stages are structurally similar even with different field names.

PHASE 16 — End-to-End Pipeline Assembly [■, Mac]

Wire all components into `src/pipeline_sql.py` and `src/pipeline_nosql.py`.

SchemaLinker inference uses `src/schema_linker/infer.py` which:

1. Formats prompt with schema text
 2. Calls `ModelInterface.generate()`
 3. Retries up to 3× if output parsing fails (retry loop from SchemaRAG)
 4. Applies `src/schema_linker/fix.py` to snap predictions to real columns
-

PHASE 17 — LangGraph Router + Self-Correction [■, Mac]

Script: `src/router/langgraph_router.py`. State machine with retry on execution failure (max 3 retries, self-correction prompt on each).

PHASE 18 — Evaluation [■]

EX metric: `src/eval/exec_eval.py` (adapted from SchemaRAG `eval/exec_eval.py`).

Key improvement over naive EX: Column-order permutation awareness — `SELECT a, b` and `SELECT b, a` are treated as equivalent when result sets match under any column permutation.

Metric	Target	Notes
Spider EX (SQL)	>82%	SchemaRAG + Qwen-7B achieves 80.4%; our r=64 LoRA should close the gap
MongoDB EX (NoSQL)	>60%	TEND SMART baseline: 65.08%

Ablation (replicate SchemaRAG Table 5): Full vs w/o SchemaLinker vs w/o SAR vs w/o POSG.

CP1 Baseline: codegen-350M EX on Spider dev (~45–55%). Establishes "without schema-awareness" floor.

PHASE 19 — Error Analysis + Iteration [■]

Error taxonomy (from SchemaRAG Figure 9):

- Schema errors (34.5%): wrong table/column → retrain SchemaLinker
- Data analysis errors (40.8%): wrong aggregation → improve Generator prompt
- Other errors (24.7%): join/condition errors → check FK graph completeness

PHASE 20 — Demo + CP4 Scope [■, Mac]

20A: Streamlit demo (k=1, <8s latency)

20B: FastAPI backend

20C: SQL-to-NoSQL migration utility (reuses Phase 5B + 14B)

SECTION 7 — RISKS AND MITIGATIONS (UPDATED)

#	Risk	Likelihood	Impact	Mitigation
1	GRPO Stage 3 OOM on Colab free T4	High	High	Colab Pro A100 required; reduce G from 8→4 as fallback
2	MongoDB conversion type mismatches	High	Medium	Type coercion in converter ■ done; wta_1 UTF-8 bug ■ fixed
3	Q-MQL verification pass rate < 40%	Medium	High	Sort by complexity (1-table first); 9/10 on test run
4	CoT filtering rejects > 50%	Medium	Medium	sqlglot entity validation is more lenient than LLM validation; observed ~80% pass
5	Self-consistency k=5 too slow for demo	High	Low	Demo uses k=1 (<8s); eval uses k=5
6	MQL stage-type similarity imprecise for POSG	Medium	Low	Acceptable v1; POSG is least critical component per SchemaRAG ablation
7	SQLite→PostgreSQL dialect differences	Low	Medium	<code>sqlglot.transpile(sql, read='sqlite', write='postgres')</code> as thin wrapper
8	SAR overfits on small NoSQL corpus	Medium	Medium	2-layer 2-head architecture; dropout 0.1; early stopping
9	DeepSeek API rate limits during batch generation	Low	Low	<code>time.sleep(0.1)</code> between calls; checkpoint every 50 entries (confirmed in code)
10	SchemaRAG CoT training data not released	Confirmed	Medium	■ Generating with Phase 8A; ~\$3 via DeepSeek
11	MongoDB not available on Colab	Medium	Low	Use <code>mongomock</code> for Colab; real MongoDB for Mac dev + demo

#	Risk	Likelihood	Impact	Mitigation
12	Generator fine-tuning diverges	Low	Medium	LR=5e-5; warmup 100 steps; monitor val loss every 500 steps
13	CoT format non-compliance (no <code>[table.col]</code> brackets)	Observed + Audited	Low	Fixed: validator, entity extractor, and key_fields extractor now share one unified anchored pattern; brackets are optional in all three; bracket mismatch no longer silently passes

SECTION 8 — DESIGN DECISIONS LOG (UPDATED)

Decision	Choice	Rationale
Routing strategy	Session-based (Option A)	Simpler; no ambiguity; sufficient for capstone
Text-to-NoSQL approach	Direct generation (SMART)	TEND paper: direct 65% EX vs cascade 44% EX — 21-point gap
NoSQL dataset source	Spider SQLite → MongoDB via DeepSeek	Reuses verified 7000 Q-SQL pairs; avoids building dataset from scratch
SchemaLinker base model	Qwen/Qwen2.5-7B (general)	CoT reasoning requires general intelligence, not code syntax
Generator base model	Qwen/Qwen2.5-Coder-7B-Instruct	Best open-source SQL benchmark; Qwen-7B in paper achieves 80.4%
SQL RAG parser	sqlglot (not sqlparse)	0 failures on all 7000 Spider SQLs; typed AST nodes
POSG AST parser	sqlparse	Token-tree structure suits edit distance; not used for structural type
Structural type vector	7 dimensions (added has_set_op)	UNION/INTERSECT/EXCEPT incompatible with plain SELECT in contrastive training
LoRA rank	r=64 (not r=16 from paper)	Higher capacity; T4 still fits Qwen-7B at bf16
CoT format	**<think> tags** (SchemaRAG updated format)	Adopted from SchemaRAG's <code>script_to_COT.py</code> ; enables easier parsing
CoT entity validation	sqlglot (not second LLM call)	Free; no extra API cost; reliable for Spider SQL patterns
BM25S: build vs query time	Build-time for training; query-time for inference	Training needs pre-computed annotations; inference benefits from question-aware values
Teacher model for CoT	DeepSeek-V3 (not GPT-4o)	~10× cheaper (\$0.0003/1K vs \$0.003/1K); comparable quality on structured CoT
SAR model source	SchemaRAG <code>train_SAR.py</code> adapted	Production-grade NaN guards; tested architecture; no reinvention needed
POSG source	SchemaRAG <code>po.py</code> adapted	ASTProcessor + Pareto front already implemented and tested
EX metric	SchemaRAG <code>eval/exec_eval.py</code> adapted	Permutation-aware; column ordering handled correctly
MongoDB schema (v1)	Reference-based separate collections	Simpler v1; embedding strategy deferred to Phase 19

Decision	Choice	Rationale
GRPO reward weights	FN:-3, TP:+2, FP:-0.5, F1:+0.5	Exact values from SchemaRAG Eq. 11
CoT validation pattern	Single unified anchored pattern for validate_format + extract_cot_key_tables + key_fields extraction	Using different patterns caused stat misclassification (entity_fail vs format_fail); unified pattern prevents inconsistency

SECTION 9 — KEY NUMBERS

From SchemaRAG (performance targets):

Configuration	Spider EX	BIRD EX
SchemaRAG + GPT-4o	85.6%	68.9%
SchemaRAG + DeepSeek-V3	89.6%	68.4%
SchemaRAG + Qwen-7B	80.4%	54.5%
SchemaRAG + XiYanSQL-14B	93.3%	65.3%
w/o SAR (Qwen-7B)	72.5%	46.7%
w/o SchemaLinker (Qwen-7B)	78.1%	55.2%

Our realistic target: 80–82% Spider EX with Qwen-7B SchemaLinker + Qwen2.5-Coder-7B generator (r=64 LoRA should close the ~2pt gap from paper's r=16).

From TEND paper (NoSQL targets):

Configuration	MongoDB EX
SMART (direct Text-to-NoSQL)	65.08%
LLM SQL→NoSQL (with correct SQL)	44.76%

Our realistic target: 55–65% MongoDB EX.

SECTION 10 — CHECKPOINT DELIVERABLES (UPDATED STATUS)

CP1 Deliverables

Item	Phase	Status
Spider dataset downloaded and validated	4	■ Done
FK graph builder functional (all 166 DBs)	5A	■ Done
MongoDB collection generator (all 166 DBs)	5B	■ Done
PromptSchema built (SQL + NoSQL)	6	■ Done
SQL RAG corpus (7000 entries, 7-dim structural type)	7A	■ Done
NoSQL RAG corpus started	7B	■ In progress
CoT data generation started	8A	■ In progress
Evaluation framework (EX metric script)	18	■ Done (src/eval/exec_eval.py)

Item	Phase	Status
SchemaRAG codebase audit + all scripts adapted	—	■ Done
Baseline: codegen-350M EX on Spider dev	18C	■ Pending

CP2 Deliverables

Item	Phase	Status
CoT data complete (SQL + NoSQL)	8A/8B	■ Pending
SchemaLinker trained (SQL, all 3 stages)	9A–11A	■ Pending
SchemaLinker trained (NoSQL, all 3 stages)	9B–11B	■ Pending
SAR trained (SQL + NoSQL)	12A/12B	■ Pending
Qwen2.5-Coder-7B fine-tuned (SQL + NoSQL)	14A/14B	■ Pending
POSG implemented (SQL + NoSQL)	15A/15B	■ Done (scripts ready)
Text-to-SQL EX > 82% on Spider dev	18A	■ Pending
Text-to-NoSQL EX > 55% on MongoDB dev	18B	■ Pending

CP3 Deliverables

Item	Phase	Status
ChromaDB indices built	13	■ Pending
End-to-end pipelines assembled	16	■ Pending
LangGraph router + self-correction	17	■ Pending
Full ablation study documented	18	■ Pending
Text-to-SQL EX > 85%	18A	■ Pending
Text-to-NoSQL EX > 60%	18B	■ Pending
Error analysis + first iteration	19	■ Pending

CP4 Deliverables

Item	Phase	Status
Streamlit demo working	20A	■ Pending
FastAPI backend	20B	■ Pending
SQL-to-NoSQL migration utility	20C	■ Pending
Final evaluation report	19	■ Pending

SECTION 11 — IMMEDIATE NEXT STEPS

Current parallel work (can run simultaneously):

1. **Phase 7B** — `python scripts/build_nosql_rag_corpus.py` (running, ~20–30 min)
2. **Phase 8A** — `python scripts/build_cot_data.py` (running, ~35–45 min)

After both complete:

3. **Phase 8B** — NoSQL CoT generation (needs 7B output)
4. **Phase 9A** — Push to Colab, begin SchemaLinker Stage 1 training
5. **Error mining for Phase 10A** — run Stage-1 inference on all 7000 Spider train entries

PDF version: No PDF tools installed on this machine. To generate PDF from this file:

- VS Code: Install "Markdown PDF" extension → right-click → Export to PDF
- CLI: `brew install pandoc && brew install basicstex`, then `pandoc CodeGen_Plan_v6_DualTrack.md -o CodeGen_Plan_v6_DualTrack.pdf`
- Online: paste into markdown-to-pdf converter